PLANTILLAS DE DOCUMENTACIÓN DE SOFTWARE

Manual de programación y código fuente + manual de usuario

Material académico editable para documentar cómo está programado un sistema y cómo utilizarlo.

Asignatura	Ingenieria de Datos
Docente	Diana Marecela Toquica Rodriguez
Programa / grupo	Ingeniería de Sistemas y Computación
Proyecto	Análisis de Mercado Vehicular

Versión de la plantilla: 2.0

PLANTILLA 1. MANUAL DE PROGRAMACIÓN Y CÓDIGO FUENTE

Objetivo: explicar cómo está organizado y cómo funciona el código fuente del proyecto, de forma que otra persona pueda leerlo, comprender sus decisiones y realizar cambios con criterio.

Nombre del sistema	Análisis de Mercado Vehicular
Versión / etiqueta del código	v2.0 (Migración a Procesamiento Distribuido con PySpark y Apache Parquet)
Lenguaje(s)	Python 3.10+, SQL (PostgreSQL Dialect)
Frameworks / librerías	Apache Spark 4.0.4 (pyspark), PostgreSQL, Google Colab Drive API (google.colab.drive)
Repositorio	[URL]
Autores	**Daniel Felipe González Rodríguez** Juan Sebastián Lozano Calderón
Fecha	31/10/2026

Control de cambios

Versión / commit	Fecha	Autor	Cambios principales
1.0	2026-08-25	Daniel González	Estructuración inicial del documento y definición de arquitectura por capas.
1.1	2026-08-25	Juan Lozano	Integración del modelo en estrella y script de perfilamiento inicial en Pandas
1.2	2026-08-29	Equipo	Consolidación del pipeline básico RAW a STAGING y repositorio en GitHub.
2.0	2026-10-30	Daniel González	Migración integral a **PySpark 4.0.4**. Ingesta tolerante a fallos.

1. Propósito y alcance del programa

El mercado de arrendamiento y venta de vehículos genera un alto volumen de información dispersa en diferentes formatos. Existe la necesidad de analizar tendencias de precios y características vehiculares unificando múltiples fuentes (ventas locales, internacionales y registros de `Craigslist`). Analistas de mercado y sistemas de tableros de control (`Dashboards`) que requieren la información centralizada y limpia para la toma de decisiones estratégicas de precios. Este sistema implementa un pipeline distribuido de Ingeniería de Datos que centraliza, perfila, normaliza y limpia tres fuentes heterogéneas extraídas de Kaggle (`vehicles.csv`, `car_sales_data_2018_10.csv` y `all_regions.csv`) que totalizan **1.933.610** registros brutos, garantizando su entrega hacia un Data Warehouse dimensional en PostgreSQL.

Funcionalidades implementadas en esta versión (v2.0):

- Inicialización del entorno de computación distribuida en clúster virtual con Apache Spark 4.0.4, configurando 8 GB de memoria de trabajo para el nodo Driver.
- Ingesta paralela tolerante a fallos con el modo `DROPMALFORMED` y soporte multilínea, eliminando bloqueos por filas corruptas o bytes no decodificables.
- Diagnóstico distribuido de calidad en la Capa RAW evaluando dimensiones, esquemas y matrices de nulos sobre 1.933.610 registros sin cargar datasets pesados en la memoria local.
- Ejecución automática de las reglas del Contrato de Datos: validación estricta de `precio > 0` y `marca NOT NULL`, lo que filtró exactamente 49.080 filas inservibles en `vehicles.csv` y consolidó 1.884.530 registros útiles en la Capa STAGING.

- Normalización y traducción controlada de términos vehiculares en ruso hacia el inglés mediante expresiones vectorizadas de mapeo (`F.create_map`), suprimiendo el riesgo de archivos en blanco o cuelgues de ejecución.
- Persistencia de alta eficiencia en formato columnar **Apache Parquet** con particionamiento y compresión nativa (`STAGING_all_regions_ing.parquet`), sustituyendo archivos CSV concatenados que llegaban a pesar 6 GB.
- Extracción analítica de la dimensión maestra `dim_vehiculo`, consolidando **325 combinaciones únicas** con claves subrogadas generadas mediante funciones de ventana analítica (`row_number()`).
- Modelado físico en DDL para PostgreSQL (`dim_vehiculo, dim_tiempo` y `fact_eventos_vehiculos`).

Funcionalidades no incluidas en esta versión:

- No incorpora interfaz gráfica de usuario (GUI) ni manuales interactivos de usuario final, ya que el sistema es un pipeline analítico de backend concebido para transferir datos depurados hacia analistas y herramientas de Business Intelligence.

Mostrar la organización de carpetas y archivos. Para cada elemento relevante, explicar qué responsabilidad tiene.

Carpeta / archivo	Tipo de elemento	Responsabilidad	Relación con otros archivos
`.venv/`	Directorio de dependencias	Aísla el entorno virtual local de Python con las versiones requeridas.	Suministra el entorno de ejecución para scripts auxiliares.
`RAW/`	Directorio de datos crudos	Almacena los 3 datasets originales de Kaggle (`vehicles.csv`, `car_sales_data_2018_10.csv` y `all_regions.csv`) sin alteración alguna.	Fuente de ingesta de sólo lectura consumida por el notebook distribuido.
`RAW/ing_datos_proyecto.ipynb`	Directorio de datos crudos	Contiene los 6 bloques ejecutables del pipeline: inicialización, perfilamiento RAW, contrato ETL, normalización Parquet, dimensión vehículo y auditoría STAGING.	Procesa los archivos de `RAW/`, genera los datasets limpios en `STAGING/` y mapea las tablas para `ANALYTICS/`.
`STAGING/STAGING_all_regions_ing.parquet`	Dataset columnar comprimido	Directorio Parquet que almacena los 1.294.757 registros normalizados de la fuente regional en cirílico traducido.	Sustituye al archivo inestable `all_regions_ing.csv` (6 GB) y sirve como entrada para la capa analítica.
`ANALYTICS/database_cars.sql`	Script DDL (PostgreSQL)	Contiene las sentencias `CREATE TABLE` con llaves primarias y foráneas del esquema en estrella.	Materializa la persistencia relacional que recibe a `dim_vehiculo` y la tabla de hechos.
`README.md`	Documentación técnica	Documenta la arquitectura, el diseño dimensional y los accesos a Google Drive y Google Colab.	

Ejemplo de estructura :

```
Project_Cars_Data_Engineering/
├── .venv/
├── ANALYTICS/
│   └── database_cars.sql
├── RAW/
│   ├── all_regions.csv
│   ├── car_sales_data_2018_10.csv
│   ├── vehicles.csv
│   └── ing_datos_proyecto.ipynb
├── STAGING/
│   └── STAGING_all_regions_ing.parquet/
│       ├── _SUCCESS
│       └── part-*.parquet
├── README.md
└── Taller_Corte_1_Semana_3_Proyecto_Gonzalez_D...
```

3. Arquitectura lógica y flujo de ejecución

3.1 Flujo general

Paso 1 (Arranque del Driver Distribuido): Se crea la sesión `SparkSession` con 8 GB de RAM asignada al driver y se conecta el almacenamiento en la nube mediante `drive.mount('/content/drive')`.

Paso 2 (Ingesta Paralela y Diagnóstico de Calidad RAW): Se cargan de forma perezosa (*Lazy Evaluation*) los tres archivos CSV aplicando `mode="DROPMALFORMED"`. Se ejecuta la función `cargar_y_diagnosticar_raw()` que calcula el total de **1.933.610 registros brutos** (**426.880** en `vehicles.csv`, **211.973** en `car_sales_data_2018_10.csv` y **1.294.757** en `all_regions.csv`) junto con sus matrices de nulidad.

Paso 3 (Filtrado por Contrato de Calidad en STAGING): Se aplica la función `limpiar_segun_contrato()` imponiendo `precio > 0` y `marca NOT NULL`. Se descartan **49.080 registros inválidos** pertenecientes a `vehicles.csv`, consolidando **1.884.530 registros útiles** en STAGING.

Paso 4 (Normalización de Encoding y Persistencia Parquet): Se eliminan copias corruptas previas en disco (`all_regions_ing.csv`) y se mapean los términos de transmisión y combustible del ruso al inglés con `F.create_map()`. El resultado se escribe con `mode("overwrite")` en formato **Parquet**.

Paso 5 (Construcción de la Dimensión Maestra): A partir del dataset limpio de ventas 2018 se extraen las marcas, modelos y años únicos con `.dropDuplicates()`, generando con `Window.orderBy` la dimensión `dim_vehiculo` con **325 registros únicos** y llaves primarias del 1 al 325.

Paso 6 (Auditoría Estadística Distribuida): Se lee el archivo Parquet de STAGING y se ejecuta `.describe()` sobre las columnas numéricas de los 1,29M de registros en pocos segundos.

3.2 Comunicación entre componentes

`google.colab.drive` → `pyspark.sql.DataFrameReader`: La API de Drive proporciona la ruta del sistema de archivos local (`/content/drive/MyDrive/ING.DATOS/`), que Spark lee dividiendo los archivos pesados en particiones gestionadas en paralelo.

Bloque 3 (`df_*_stg`) → Bloque 4 (`DataFrameWriter.parquet`): El flujo transformado de `df_regions_stg` se transmite al motor de persistencia columnar optimizada Parquet, reduciendo el consumo de disco y acelerando lecturas futuras.

Bloque 5 (`dim_vehiculo`) → Capa Analítica (`script_ddl` en PostgreSQL): La tabla en memoria generada por PySpark coincide campo a campo con los tipos y restricciones de integridad (`SERIAL PRIMARY KEY`, `VARCHAR(100) NOT NULL`) definidos en `database_cars.sql`.

4. Clases, módulos y responsabilidades

Clase / módulo	Responsabilidad	Atributos / datos principales	Dependencias
Módulo 1: Inicialización del Clúster Spark	Configurar el runtime distribuido y montar el almacenamiento persistente de Google Drive.	`spark`, `RUTA_BASE` (`"/content/drive/MyDrive/ING.DATOS"`).	`pyspark.sql.SparkSession`, `google.colab.drive`.
Módulo 2: Ingesta y Perfilamiento RAW	Leer los CSV crudos con tolerancia a líneas corruptas y cuantificar nulos, registros y esquemas.	`df_vehicles_raw`, `df_sales_raw`, `df_regions_raw`, `cargar_y_diagnosticar_raw()`.	`pyspark.sql.functions as F`.
Módulo 3: Motor del Contrato de Calidad (ETL)	Validar precio positivo, obligatoriedad de marca e imputar nulos en kilometraje negativo.	`limpiar_segun_contrato()`, `df_vehicles_stg`, `df_sales_stg`, `df_regions_stg`.	`pyspark.sql.functions as F`.
Módulo 4: Traductor de Encoding y Escritura Parquet	Validar precio positivo, obligatoriedad de marca e imputar nulos en kilometraje negativo.	`diccionario_ruso_ingles`, `mapping_expr`, `ruta_parquet_stg`.	`itertools.chain`, `pyspark.sql.functions.`
Módulo 5: Generador de Dimensión Vehículo	Aislar atributos físicos, eliminar redundancias y numerar claves primarias analíticas.	`dim_vehiculo`, `ventana_id`, `script_ddl`	`pyspark.sql.window.Window`
Módulo 6: Inspector Estadístico de STAGING	Aislar atributos físicos, eliminar redundancias y numerar claves primarias analíticas.	`df_verificacion`, columnas `['price', 'year', 'mileage']`.	`pyspark.sql.DataFrame.describe()`

4.1 Ficha de clase o módulo

Ficha 1: Ingesta Tolerante y Diagnóstico RAW (`cargar_y_diagnosticar_raw`)

Nombre	`cargar_y_diagnosticar_raw`
Archivo	Bloque 2 de `ing_datos_proyecto.ipynb`
Propósito	Ingerir fuentes masivas de hasta 2 GB descartando filas malformadas y auditar valores nulos de forma distribuida.
Se relaciona con	Datasets físicos en la Capa RAW y DataFrames crudos (`df_vehicles_raw`, etc.).

Atributos y variables clave:

- `ruta_archivo` (`str`): Ubicación del CSV crudo en Drive.
- `nombre_fuente` (`str`): Etiqueta descriptiva para trazabilidad en logs.
- `expresiones_nulos` (`list`): Lista de agregaciones F.count(F.when(...)) construida dinámicamente verificando tipos string para comparar cadenas vacías sin provocar fallos de casteo numérico.

Métodos invocados:

- `spark.read.csv(..., mode="DROPMALFORMED", multiLine=True)`: Ingesta paralela tolerante a bytes corruptos.
- `df.select(expresiones_nulos).show(vertical=True)`: Cálculo y presentación vertical de la matriz de nulidad.

Ficha 2: Motor del Contrato de Calidad (`limpiar_segun_contrato`)

Nombre	`limpiar_segun_contrato`
Archivo	Bloque 3 de `ing_datos_proyecto.ipynb`
Propósito	Aplicar las reglas de obligatoriedad y validez del Contrato de Datos antes de enviar registros a la Capa STAGING.
Se relaciona con	Tablas de la Capa RAW y generación de los DataFrames depurados de STAGING.

Atributos y variables clave:

- `col_precio` (`str`): Nombre del atributo de precio según la fuente ("price" o "Sale Price").
- `col_marca` (`str`): Nombre del atributo de fabricante ("manufacturer", "Car Make" o "brand").
- `col_km` (`str`, `opcional`): Nombre de la columna de kilometraje ("odometer" o "mileage").

Métodos invocados:

- `df.withColumn(col_precio, F.col(col_precio).cast("double")).filter(...)`: Filtrado estricto donde precio > 0.
- `df.filter(F.col(col_marca).isNotNull() & (F.trim(F.col(col_marca)) != ""))`: Rechazo inmediato de marcas no identificadas.
- `F.when(F.col(col_km) >= 0, F.col(col_km)).otherwise(F.lit(None))`: Imputación de nulo para valores de kilometraje ilógicos o negativos.

5. Explicación de funciones, métodos y algoritmos

5.1 Algoritmo de Ingesta y Diagnóstico Distribuido de la Capa RAW

Archivo y ubicación: `ing_datos_proyecto.ipynb` — Bloque 2.

Objetivo: Leer 1,93 millones de filas distribuidas en archivos de 20 MB a 2 GB sin detener la ejecución ante líneas corruptas y cuantificar el estado de calidad.

Entradas y salidas

Parámetros: `ruta_archivo` (ruta del archivo CSV) y `nombre_fuente` (identificador en consola).

Retorno: Tupla (`df, total_filas`) que contiene el DataFrame de PySpark inicializado y el conteo de filas válidas.

Efectos: Imprime en consola el conteo de filas, columnas, esquema de datos y la cantidad exacta de nulos por atributo.

Lógica paso a paso

1. Se invoca `spark.read.csv` configurando `escape='"'`, `multiLine=True` y `mode="DROPMALFORMED"` para descartar en lectura cualquier línea con número de campos descuadrado o caracteres binarios.

2. Se evalúa el tamaño de la matriz con `df.count()` y `len(df.columns)`.

3. Se crea un diccionario de tipos con `dict(df.dtypes)` para aislar las columnas de tipo `string`.

4. Para cada columna, se construye una condición en Spark SQL que evalúa si el valor es nulo (`isNull()`), o si siendo string es una cadena vacía (`== ""`), evitando conversiones de tipos inválidas sobre números.

5. Se ejecuta la agregación en un único pase de cálculo distribuido y se despliega con `show(vertical=True)`.

Fragmento de código

```
def cargar_y_diagnosticar_raw(ruta_archivo, nombre_fuente):
    print(f"\n==========================================")
    print(f"--- Diagnóstico Básico Capa RAW: {nombre_fuente} ---")

    df = spark.read.csv(
        ruta_archivo,
        header=True,
        inferSchema=True,
        multiLine=True,
        escape='"',
        encoding="UTF-8",
        mode="DROPMALFORMED"
    )

    total_filas = df.count()
    total_columnas = len(df.columns)
    print(f"Total de filas leídas válidas: {total_filas:,} | Total de columnas:
{total_columnas}")

    tipos_columnas = dict(df.dtypes)
    expresiones_nulos = [
        F.count(
            F.when(
                F.col(c).isNull() | (F.lit(tipos_columnas[c] == "string") &
(F.col(c) == "")), c
            )
        ).alias(c)
        for c in df.columns
    ]
    df.select(expresiones_nulos).show(vertical=True)
    df.printSchema()
  return df, total_filas
```

Explicación del código

l uso de `DROPMALFORMED` actúa como escudo de seguridad a nivel de motor de datos: en vez de lanzar excepciones fatales al encontrar caracteres binarios o retornos de carro huérfanos, descarta la fila a nivel de partición. La evaluación condicional por tipo de dato previene el fallo clásico de PySpark al comparar una columna entera con `""`.

Ejemplo de ejecución

Entrada: `car_sales_data_2018_10.csv`.

Proceso: Lectura perezosa, verificación de 9 columnas y conteo de nulos.

Salida obtenida`: Total de filas leídas válidas: 211,973 | Total de columnas: 9`. Todos los campos con 0 nulos exactos.

Casos especiales y errores

Si una columna numérica contiene valores de texto corruptos, `inferSchema` la interpretará como `string`. El Módulo 3 subsana esto forzando `.cast("double")`.

5.2 Algoritmo de Limpieza bajo Contrato de Datos (Capa RAW a STAGING)

Archivo y ubicación: `ing_datos_proyecto.ipynb` — Bloque 3.

Objetivo: Filtrar los 1.933.610 registros brutos descartando registros inválidos para generar la Capa STAGING.

Entradas y salidas

- **Parámetros:** `df` (DataFrame crudo), `col_precio` (nombre de columna precio), `col_marca` (nombre de columna marca) y `col_km` (opcional, columna odómetro).
- **Retorno:** `DataFrame` filtrado y tipificado.
- **Efectos:** Reduce el volumen global descartando las 49.080 filas inválidas de `vehicles.csv`.

Lógica paso a paso

1. Convierte el campo de precio a `double` mediante `cast("double")` y aplica `.filter()` requiriendo que no sea nulo y que sea estrictamente mayor a 0.
2. Aplica un segundo filtro sobre la columna de marca asegurando que no sea nula ni contenga cadenas de texto en blanco (`trim != ""`).
3. Si existe columna de kilometraje, la castea a `double` y mediante `F.when(col >= 0, col).otherwise(None)` reemplaza cualquier kilometraje negativo por `NULL` sin perder la transacción.

Fragmento de Código

```
def limpiar_segun_contrato(df, col_precio, col_marca, col_km=None):
    df_limpio = df.withColumn(col_precio, F.col(col_precio).cast("double")) \
                  .filter(F.col(col_precio).isNotNull() & (F.col(col_precio) > 0))

    df_limpio = df_limpio.filter(
        F.col(col_marca).isNotNull() & (F.trim(F.col(col_marca)) != "")
    )

    if col_km and col_km in df_limpio.columns:
        df_limpio = df_limpio.withColumn(col_km, F.col(col_km).cast("double"))
        df_limpio = df_limpio.withColumn(
            col_km,
```

```
            F.when(F.col(col_km) >= 0, F.col(col_km)).otherwise(F.lit(None))
        )
    return df_limpio
```

Explicación del código

Aplica directamente las reglas de obligatoriedad del Contrato de Datos unificado. Permite procesar de forma uniforme esquemas distintos (`price` vs `Sale Price`, `manufacturer` vs `brand`) garantizando que ningún registro huérfano de precio o fabricante pase a la base de datos.

Ejemplo de ejecución real

- **Entrada:** `vehicles.csv` con 426.880 filas brutas.
- **Proceso:** Filtrado de filas con precio cero/negativo y 17.646 marcas nulas.
- **Salida obtenida:** Sobreviven exactamente **377.800 filas válidas**; se descartan **49.080 registros inservibles**.

5.3 Algoritmo de Traducción Vectorizada y Persistencia Parquet

- **Archivo y ubicación:** `ing_datos_proyecto.ipynb` — Bloque 4.
- **Objetivo:** Resolver los problemas de caracteres en cirílico ruso de `all_regions.csv` traduciendo términos categóricos clave sin usar APIs lentas, y persistir los 1,29M de registros en Apache Parquet.

Entradas y salidas

- **Parámetros:** Diccionario `diccionario_ruso_ingles`, DataFrame `df_regions_stg`, ruta destino Parquet.
- **Retorno:** Directorio `/content/drive/MyDrive/ING.DATOS/STAGING_all_regions_ing.parquet`.
- **Efectos:** Elimina el archivo corrupto anterior de 6 GB en Drive y genera el almacenamiento columnar definitivo.

Lógica paso a paso

1. Se define un diccionario con las traducciones canónicas de transmisiones y combustibles (`"механика": "manual"`, `"автомат": "automatic"`, etc.).
2. Se compila en una expresión distribuida de Spark mediante `F.create_map([F.lit(x) for x in chain(*items)]).`
3. Se aplica sobre las columnas categóricas con `F.coalesce(mapping_expr[F.lower(col)], col)`, de modo que si un término no está en el mapa, conserva su valor original sin corromperse.
4. Se verifica si existe el archivo `.csv` sobredimensionado previo con `os.path.exists()` y se elimina con `os.remove().`
5. Se invoca `.write.mode("overwrite").parquet()`, ejecutando la persistencia distribuida en paralelo.

Fragmento de código

```
from itertools import chain

diccionario_ruso_ingles = {
    "механика": "manual", "автомат": "automatic", "робот": "robotized",
    "вариатор": "cvt", "бензин": "gasoline", "дизель": "diesel",
    "гибрид": "hybrid", "электро": "electric", "газ": "gas",
    "передний": "fwd", "задний": "rwd", "полный": "awd"
}

mapping_expr = F.create_map([F.lit(x) for x in chain(*diccionario_ruso_ingles.items())])

columnas_traducir = ["transmission", "engine_fuel", "fuel", "drivetrain", "drive"]
for col_name in columnas_traducir:
    if col_name in df_regions_stg.columns:
        df_regions_stg = df_regions_stg.withColumn(
            col_name,
            F.coalesce(mapping_expr[F.lower(F.col(col_name))], F.col(col_name))
        )

archivo_csv_viejo = f"{RUTA_BASE}/all_regions_ing.csv"
if os.path.exists(archivo_csv_viejo) and os.path.isfile(archivo_csv_viejo):
    os.remove(archivo_csv_viejo)

ruta_parquet_stg = f"{RUTA_BASE}/STAGING_all_regions_ing.parquet"
df_regions_stg.write.mode("overwrite").parquet(ruta_parquet_stg)
```

Explicación del código

El mapeo vectorizado con `F.create_map` se ejecuta dentro del motor Catalyst de Spark en C++/JVM sin transferir datos a Python. La escritura en Parquet organiza los datos por columnas comprimidas mediante Snappy, reduciendo el tamaño en disco de más de 2 GB a menos de 300 MB y evitando que el archivo vuelva a corromperse.

5.4 Algoritmo de Extracción y Deduplicación Dimensional (dim_vehiculo)

- **Archivo y ubicación:** `ing_datos_proyecto.ipynb` — Bloque 5.
- **Objetivo:** Generar el catálogo de vehículos sin redundancias y asignar claves subrogadas correlativas.

Entradas y salidas:

- **Parámetros:** `df_sales_stg` (211.973 registros limpios).
- **Retorno:** `DataFrame dim_vehiculo` con 325 registros y 4 columnas (`id_vehiculo`, `marca`, `modelo`, `anio_fabricacion`).
- **Efectos:** Despliega los primeros 10 vehículos catalogados y genera el catálogo listo para persistencia analítica.

Lógica paso a paso:

1. Proyecta los campos físicos (`Car Make`, `Car Model`, `Car Year`) desde `df_sales_stg`.
2. Aplica `.dropDuplicates()` colapsando transacciones repetidas.
3. Renombra a los identificadores del Data Warehouse (`marca`, `modelo`, `anio_fabricacion`).
4. Define una especificación de ventana `Window.orderBy("marca", "modelo", "anio_fabricacion")`.
5. Aplica `F.row_number().over(ventana_id)` para asignar `id_vehiculo` secuencial del 1 al 325.

Fragmento de código:

```
dim_vehiculo = df_sales_stg.select("Car Make", "Car Model", "Car Year") \
                    .dropDuplicates()

dim_vehiculo = dim_vehiculo.withColumnRenamed("Car Make", "marca") \
                    .withColumnRenamed("Car Model", "modelo") \
                    .withColumnRenamed("Car Year", "anio_fabricacion")

ventana_id = Window.orderBy("marca", "modelo", "anio_fabricacion")
dim_vehiculo = dim_vehiculo.withColumn("id_vehiculo", F.row_number().over(ventana_id)) \
                    .select("id_vehiculo", "marca", "modelo", "anio_fabricacion")
```

Explicación del código:

Garantiza idempotencia: al ordenar por atributos antes de generar `row_number()`, el identificador `id_vehiculo` asignado a cada vehículo es determinista y no depende del particionamiento interno de Spark.

Ejemplo de ejecución real:

- **Entrada:** 211.973 transacciones.
- **Salida obtenida:** Exactamente **325 vehículos únicos** catalogados. Muestra: `id_vehiculo` = `1` a `10` correspondiente a Chevrolet Altima años 2010 a 2019.

6. Variables, estructuras de datos y reglas de negocio

Variable / estructura	Tipo	Dónde se utiliza	Por qué se utiliza / regla
`spark`	`SparkSession`	Bloques 1 a 6 de `ing_datos_proyecto.ipynb`	Punto de entrada del clúster PySpark en Colab; gestiona la memoria y el planificador distribuido.
`df_*_raw`	`pyspark.sql.DataFrame`	Bloque 2	Estructuras distribuidas de lectura inmutable para la Capa RAW (1,93M de filas).
`df_*_stg`	`pyspark.sql.DataFrame`	Bloques 3 y 4	Vistas depuradas que cumplen el Contrato de Calidad (1.884.530 filas en total).
`dim_vehiculo`	`pyspark.sql.DataFrame`	Bloque 5	Tabla dimensional con 325 registros únicos y clave primaria entera secuencial.
`STAGING_all_regions_ing.parquet`	Archivo físico Parquet	Almacenamiento Google Drive / STAGING	Estructura columnar con compresión Snappy que preserva tipos nativos (`double`, `string`) sin truncamiento.
`script_ddl`	`str` (SQL)	Bloque 5 `/ANALYTICS/database_cars.sql`	Define las restricciones relacionales `PRIMARY KEY`, `NOT NULL` y `REFERENCES` para PostgreSQL.

Reglas de negocio del Contrato de Datos unificado

Volumen consolidado: Ingesta de 1.933.610 registros brutos en RAW →descarte de 49.080 registros corruptos/inválidos →consolidación de 1.884.530 registros útiles en STAGING.

Completitud y precio: Todo vehículo debe tener `precio > 0` y `marca NOT NULL`; ante cualquier omisión, la fila se rechaza.

Consistencia de kilometraje: Valores negativos se imputan como `NULL` conservando la venta.

Integridad dimensional: Cada venta en la tabla de hechos debe referenciar a un registro válido en `dim_vehiculo`.

7. Validaciones y manejo de errores

Validación o error	Dónde se controla	Comportamiento del código	Mensaje / resultado
Filas truncadas o bytes corruptos	Bloque 2 (`mode="DROPMALFORMED"`)	Spark descarta en el análisis sintáctico cualquier línea con número erróneo de columnas o bytes no UTF-8.	Lectura sin excepciones; asegura que el DataFrame sea procesable.
Precio nulo `o ≤ 0`	Bloque 3 (`limpiar_segun_contrato`)	Convierte a double y filtra `col_precio > 0`.	**Rechazar registro:** Se filtran filas inválidas de `vehicles.csv`.
Marca nula o vacía	Bloque 3 (`limpiar_segun_contrato`)	Filtra `isNotNull()` y `trim != ""`.	**Rechazar registro:** Elimina 17.646 nulos de vehicles.csv.
Kilometraje negativo	Bloque 3 (`limpiar_segun_contrato`)	Aplica condicional `F.when(km >= 0, km).otherwise(None)`.	**Imputar nulo:** Mantiene el registro sin corromper el cálculo de promedios
Duplicación de peso a 6 GB por `mode='a'`	Bloque 4 (`os.remove` + Parquet `overwrite`)	Elimina el CSV residual previo y escribe Parquet con sobreescritura atómica.	Almacenamiento limpio y estable en Drive.
Casteo inválido en auditoría de nulos	Bloque 2 (`tipos_columnas[c] == "string"`)	Aplica la comparación `== ""` exclusivamente a columnas de texto.	Previene fallos en Spark al auditar campos numéricos.

8. Persistencia y manejo de datos

Operación	Archivo / método responsable	Datos utilizados	Resultado
Lectura distribuida (Capa RAW)	`ing_datos_proyecto.ipynb` / `spark.read.csv()`	`vehicles.csv` (1.35 GB), car_sales_data_2018_10.csv (20 MB), all_regions.csv (2 GB).	**1.933.610 registros brutos** cargados de forma perezosa.
Persistencia optimizada (Capa STAGING)	`ing_datos_proyecto.ipynb` / `df.write.parquet()`	1.294.757 registros de `all_regions` traducidos y depurados.	Carpeta `STAGING_all_regions_ing.parquet` en Google Drive (~250 MB).
Definición relacional (Capa Analítica)	`script_ddl` en Bloque 5 / `database_cars.sql`	Esquema en estrella: `dim_vehiculo`, `dim_tiempo` y `fact_eventos_vehiculos`.	Esquema preparado para inserción de los 1.884.530 registros limpios en PostgreSQL.

Script DDL documentado para PostgreSQL (`ANALYTICS/database_cars.sql`):

```sql
CREATE TABLE dim_vehiculo (
    id_vehiculo SERIAL PRIMARY KEY,
    marca VARCHAR(100) NOT NULL,
    modelo VARCHAR(100),
    anio_fabricacion INT,
    estado_condicion VARCHAR(50),
    tipo_carroceria VARCHAR(100),
    transmision VARCHAR(50)
);

CREATE TABLE dim_tiempo (
    id_tiempo INT PRIMARY KEY,
    fecha_completa DATE NOT NULL,
    anio INT,
    mes INT
);

CREATE TABLE fact_eventos_vehiculos (
    id_evento SERIAL PRIMARY KEY,
    id_vehiculo INT REFERENCES dim_vehiculo(id_vehiculo),
    id_tiempo INT REFERENCES dim_tiempo(id_tiempo),
    precio NUMERIC NOT NULL,
    kilometraje NUMERIC
);
```

9. Interfaces, APIs o servicios externos

Componente / endpoint	Método que lo invoca	Datos enviados	Respuesta y manejo de errores
Google Drive API (`google.colab.drive`)	`drive.mount('/content/drive')`	Parámetro `/content/drive`.	Monta la unidad; detecta si ya estaba montada (`Drive already mounted`) evitando advertencias de reconexión.
Motor Apache Spark 4.0.4	`SparkSession.builder...getOrCreate()`	Parámetros de memoria del driver (`8g`) y compatibilidad temporal (`LEGACY`).	Inicia el contexto de cómputo en memoria con acceso a núcleos paralelos.

10. Pruebas relacionadas con el código

Prueba	Método / componente	Entrada	Resultado esperado	Resultado obtenido
Diagnóstico RAW 1: `vehicles.csv`	Bloque 2 (`cargar_y_diagnosticar_raw`)	`vehicles.csv` (~1.35 GB)	Leer ~426K filas y detectar nulos.	**426.880 filas leídas válidas, 26 columnas**. Nulos: `year` = 1.205, `manufacturer` = 17.646, `condition` = 174.104, `county` = 426.880.
Diagnóstico RAW 2: `car_sales_2018`	Bloque 2 (`cargar_y_diagnosticar_raw`)	`car_sales_data_2018_10.csv` (~20 MB)	Confirmar calidad óptima de la fuente.	**211.973 filas leídas válidas, 9 columnas**. `0` nulos en todas las columnas.

Diagnóstico RAW 3: `all_regions.csv`	Bloque 2 (`cargar_y_diagnosticar_raw`)	`all_regions.csv` (~2 GB)	Leer 1,29M de filas sin desbordar memoria.	1.294.757 filas leídas válidas, 18 columnas. Nulos: `year` = 570.113, `mileage` = 522.958, `price` = 0.
Volumen Bruto Total RAW	Bloque 2 (Suma de conteos)	3 fuentes combinadas	Totalizar ~1.93M de registros.	1.933.610 registros diagnosticados en RAW.
Contrato de Calidad (STAGING)	Bloque 3 (`limpiar_segun_contrato`)	1.933.610 registros	Filtrar precios ≤0 y marcas nulas.	1.884.530 registros útiles consolidados. Se descartaron 49.080 registros inservibles (todos de `vehicles.csv`).
Extracción `dim_vehiculo`	Bloque 5 (`dropDuplicates` + Window)	211.973 transacciones	Generar catálogo sin duplicados con PK.	325 vehículos únicos catalogados. Llaves del 1 al 325 asignadas exitosamente.
Auditoría Estadística Parquet STAGING	Bloque 6 (`describe()` sobre Parquet)	`STAGING_all_regions_ing.parquet`	Obtener estadísticas de precio, año y kilometraje.	Matriz calculada en segundos: ing_datos_proyecto.ipynb • Precio: `media` = `$1.444.357,82`, `mín` = `$270`, `máx` = `$150.000.000.` • Año: `media` = `2009,68`, `mín` = `1936`, `máx` = `2023`. • Kilometraje: `media` = `154.893,40km`, `mín` = `1.000km`, `máx` = `1.000.000km`.

10.1 Casos límite

✓ **Valores mínimos y máximos:** Precios de $270 a $150.000.000 y años de 1936 a 2023 procesados sin truncamiento numérico mediante tipo `double`.

✓**Entradas nulas o vacías masivas:** Control de los 570.113 nulos en año y 522.958 en kilometraje de `all_regions.csv`.

✓ **Deduplicación:** Reducción exitosa de 211.973 ventas a 325 modelos únicos en `dim_vehiculo`.

✓ **Gestión de archivos de gran volumen:** Solución definitiva al error de crecimiento a 6 GB mediante sobreescritura atómica en formato Parquet.

11. Decisiones de implementación

Migración de Pandas monohilo a Apache Spark 4.0.4: Pandas colapsaba la memoria RAM de Colab al procesar los CSV de 1.35 GB y 2 GB. Spark permite evaluar transformaciones de forma perezosa y paralelizar el procesamiento en múltiples hilos con tolerancia a fallos.

Adopción de Apache Parquet sobre CSV plano en STAGING: Los CSV crecían de forma descontrolada hasta 6 GB con el modo `append`. Parquet almacena datos de forma columnar con compresión Snappy, permitiendo que consultas estadísticas (`describe`) solo lean los bytes de las columnas involucradas.

Traducción por diccionario vectorizado en lugar de APIs por fila: Traducir 1,29 millones de registros fila por fila bloqueaba el entorno. El uso de `F.create_map` con normalización de términos vehiculares ejecuta la traducción en milisegundos en memoria.

12. Limitaciones y mejoras pendientes

Deuda técnica resuelta en esta versión: Se corrigió el orden de montaje de Google Drive, se eliminó la duplicación descontrolada de archivos temporales (resolviendo el archivo de 6 GB) y se evitó el desbordamiento de memoria RAM al migrar la limpieza y auditoría a PySpark y Apache Parquet.

Mejoras pendientes (Hacia Corte 2 y Cierre):

1. **Carga masiva hacia PostgreSQL:** Implementar la escritura mediante el conector JDBC de PySpark (`df.write.jdbc(...)`) para poblar directamente las tablas `dim_vehiculo` y `fact_eventos_vehiculos` definidas en `database_cars.sql`.
2. **Refinamiento de normalización de texto:** Continuar con la estandarización de términos categóricos restantes (como tipos de carrocería y colores específicos) que aún conservan caracteres en cirílico en `all_regions.csv`.
3. **Optimización de índices y particionamiento en base de datos:** Crear índices sobre las claves foráneas (`id_vehiculo`, `id_tiempo`) en PostgreSQL para acelerar las consultas agregadas que consumirán los tableros de BI.

13. Cómo ejecutar y revisar el código

1. **Requisitos previos:** Cuenta en Google Colab con Google Drive conectado (al menos 8 GB libres) y entorno con Java Virtual Machine (preinstalado en Colab para PySpark).
2. **Obtener el código y datos:**

   Clonar repositorio:
   ```
   git clone
   https://github.com/jlozano1118/Project_Cars_Data_Engineering
   ```
   Guardar los datasets en Google Drive dentro de la carpeta `/content/drive/MyDrive/ING.DATOS/`.
3. **Ejecución en Colab:**
   Abrir ing_datos_proyecto.ipynb.
   Ejecutar secuencialmente los Bloques 1 a 6.

4. **Verificación de resultados:**

   Comprobar en Bloque 2 el diagnóstico de **1.933.610 registros brutos**.

   Comprobar en Bloque 3 la consolidación de **1.884.530 registros útiles** y el descarte de las **49.080 filas inválidas**.

   Comprobar en Bloque 5 la generación de los **325 vehículos únicos** en `dim_vehiculo`.

   Validar en Bloque 6 el resumen estadístico de `STAGING_all_regions_ing.parquet`.

14. Glosario y referencias

Término técnico	Definición aplicada al proyecto
Apache Spark / PySpark	Motor de procesamiento analítico distribuido en clúster utilizado para transformar en paralelo los 1,93 millones de registros vehiculares.
Apache Parquet	Formato de archivo binario columnar con compresión Snappy utilizado en la Capa STAGING para evitar archivos corruptos o sobredimensionados
`DROPMALFORMED`	Modo de lectura en PySpark que descarta de inmediato cualquier registro con delimitadores o codificación dañada sin abortar la ejecución.
Contrato de Calidad de Datos	Conjunto de reglas obligatorias (`precio > 0`, `marca NOT NULL`) que depuraron 49.080 registros inservibles antes de la persistencia analítica.
Esquema Estrella (*Star Schema*)	Paradigma de modelado dimensional que desacopla hechos cuantitativos (`fact_eventos_vehiculos`) de descriptores de negocio (`dim_vehiculo`, `dim_tiempo`), minimizando la cantidad de `JOIN` en consultas analíticas.
Llave Subrogada (`id_vehiculo`)	Clave primaria artificial secuencial generada con funciones de ventana (`row_number()`) para conectar dimensiones con hechos.
Funciones de Ventana (*Window Functions*)	Operaciones analíticas en Spark SQL (`pyspark.sql.window.Window`) que permiten calcular agregaciones o numeraciones correlativas (`row_number()`) sobre particiones de datos sin colapsar las filas.
`UTF-8-SIG` (BOM)	Estándar de codificación Unicode que antepone la Marca de Orden de Bytes (*Byte Order Mark*, `0xEF`, `0xBB`, `0xBF`), garantizando que caracteres internacionales (como cirílico ruso o tildes) se interpreten correctamente en distintos sistemas operativos.